

Практическая работа №3

Тема:

Работа с данными и табличными представлениями. Интеграция Spring Data JPA и Vaadin Grid

Цель: превратить класс Document в полноценную сущность БД, создать репозиторий для управления данными и реализовать страницу со списком документов и функцией поиска.

Краткие теоретические сведения

JPA (Java Persistence API) - спецификация, которая описывает, как Java-объекты отображаются на таблицы в базе данных (ORM - Object-Relational Mapping).

Entity (Сущность) - обычный Java-класс, помеченный аннотацией @Entity. Каждое поле такого класса соответствует колонке в таблице БД.

Repository (Репозиторий) - интерфейс, который предоставляет готовые методы для работы с данными: поиск (findById, findAll), сохранение (save) и удаление (delete). Spring автоматически генерирует реализацию этих методов во время запуска приложения.

Vaadin Grid - мощный компонент для отображения списков данных в виде таблиц. Поддерживает ленивую загрузку (загружает из базы только те строки, которые видны на экране), сортировку по колонкам и фильтрацию.

Derived Query Methods - методы репозитория, имя которых описывает запрос (например, findByTitleContainingIgnoreCase). Spring Data JPA генерирует JPQL по имени метода автоматически.

Задание:

Часть 1. Подготовка проекта к работе с БД

Откройте проект, созданный в Практической работе №2.

Убедитесь, что в `pom.xml` есть зависимости для JPA и PostgreSQL. В проекте, созданном через `start.vaadin.com`, они обычно уже присутствуют. Если нет - добавьте:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Откройте файл `src/main/resources/application.properties` и добавьте параметры подключения к БД (из Практической работы №1). Пример, если PostgreSQL запущен в Docker с вашим пользователем и паролем.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/atomdb
spring.datasource.username=<вашлогин>
spring.datasource.password=<вашпароль>
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

Пояснения:

ddl-auto=update — Hibernate будет автоматически создавать/обновлять таблицы по сущностям. В реальных проектах используют *validate* или миграции (Flyway, Liquibase), но для учебных целей *update* удобно.

show-sql=true — SQL-запросы будут выводиться в консоль (удобно для отладки).

Часть 2. Превращение Document в JPA-сущность

Откройте класс `Document` в пакете `documentfeature`, созданный в Практической работе №2. Добавьте JPA-аннотации: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`. Также добавьте новое поле `id`.

```

package com.<ваша_фамилия>.documentfeature;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "documents")
public class Document {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "title", nullable = false)
    private String title;

    @Column(name = "reg_number", unique = true)
    private String regNumber;

    @Column(name = "reg_date")
    private LocalDate regDate;

    @Column(name = "type")
    private String type;

    public Document() {
    }

    public Document(String title, String regNumber, LocalDate regDate, String
type) {
        this.title = title;
        this.regNumber = regNumber;
        this.regDate = regDate;
        this.type = type;
    }

    // Геттеры и сеттеры сгенерируйте в IDE: Alt+Insert → Getter and Setter
    // Не забудьте добавить getId() и setId() для нового поля id
}

```

Обратите внимание:

Добавилось поле `id` с автогенерацией – это первичный ключ.

@Column – необязательна, но полезна для указания имен колонок и ограничений (nullable, unique).

Поля private + геттеры/сеттеры – обязательное требование POJO для JPA.

Часть 3. Создание репозитория

Создайте интерфейс DocumentRepository.java в пакете documentfeature. Обратите внимание: в Практической работе №2 у вас был класс DocumentRepository (заглушка с хранением в ArrayList). Теперь его нужно заменить на интерфейс – Spring Data JPA сам сгенерирует реализацию

```
package com.<ваша_фамилия>.documentfeature;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DocumentRepository extends JpaRepository<Document, Long> {

    // Поиск документов по части названия (без учета регистра)
    List<Document> findByTitleContainingIgnoreCase(String title);
}
```

Пояснения:

- JpaRepository<Document, Long> – базовый интерфейс Spring Data JPA. Дает готовые методы: save, findById, findAll, deleteById, count и др.
- Document – тип сущности.
- Long – тип первичного ключа.
- findByTitleContainingIgnoreCase – Derived Query. Spring Data JPA сгенерирует JPQL-запрос автоматически по имени метода.

Часть 4. Обновление сервиса

Класс DocumentService из Практической работы №2 использовал заглушку репозитория с хранением в памяти. Теперь замените его содержимое на работу с JPA-репозиторием.

```
package com.<ваша_фамилия>.documentfeature;
```

```
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class DocumentService {

    private final DocumentRepository documentRepository;

    public DocumentService(DocumentRepository documentRepository) {
        this.documentRepository = documentRepository;
    }

    public Document saveDocument(Document document) {
        return documentRepository.save(document);
    }

    public List<Document> findAllDocuments() {
        return documentRepository.findAll();
    }

    public List<Document> findDocumentsByTitle(String title) {
        if (title == null || title.isBlank()) {
            return findAllDocuments();
        }
        return documentRepository.findByTitleContainingIgnoreCase(title);
    }
}
```

Пояснения:

- Метод `findDocumentsByTitle` — обертка: если строка поиска пустая, возвращаем все документы; иначе — фильтруем через репозиторий.
- `documentRepository` внедряется через конструктор (Dependency Injection).

Часть 5. Создание View со списком документов

Создайте класс DocumentListView.java в пакете documentfeature.ui.

```
package com.<ваша_фамилия>.documentfeature.ui;

import com.vaadin.flow.component.grid.Grid;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.value.ValueChangeMode;
import com.vaadin.flow.router.Route;
import com.<ваша_фамилия>.base.ui.MainLayout;
import com.<ваша_фамилия>.documentfeature.Document;
import com.<ваша_фамилия>.documentfeature.DocumentService;
import org.springframework.beans.factory.annotation.Autowired;

@Route(value = "documents", layout = MainLayout.class)
public class DocumentListView extends VerticalLayout {

    private final DocumentService documentService;

    private Grid<Document> grid = new Grid<>(Document.class);
    private TextField searchField = new TextField("Поиск по названию");

    public DocumentListView(@Autowired DocumentService documentService) {
        this.documentService = documentService;

        // Настройка поискового поля
        searchField.setPlaceholder("Введите часть названия...");
        searchField.setClearButtonVisible(true);
        searchField.setValueChangeMode(ValueChangeMode.LAZY);
        searchField.addValueChangeListener(e -> updateList());

        // Настройка колонок Grid
        configureGrid();

        // Добавляем компоненты на форму
        add(searchField, grid);

        // Загружаем данные
        updateList();
    }

    private void configureGrid() {
        grid.removeAllColumns(); // убираем автоматические колонки
    }
}
```

```

        grid.addColumn(Document::getTitle).setHeader("Название");
        grid.addColumn(Document::getRegNumber).setHeader("Регистрационный
номер");
        grid.addColumn(Document::getRegDate).setHeader("Дата регистрации");
        grid.addColumn(Document::getType).setHeader("Тип документа");
        grid.setSizeFull();
    }

    private void updateList() {
        String searchTerm = searchField.getValue();
        grid.setItems(documentService.findDocumentsByTitle(searchTerm));
    }
}

```

Пояснения:

`@Route(value = "documents", layout = MainLayout.class)` – URL страницы: <http://localhost:8080/documents>. `MainLayout` — общий layout из проекта Vaadin Start (боковое меню).

`ValueChangeMode.LAZY` – событие срабатывает с задержкой, чтобы не делать запрос на каждый ввод символа.

`grid.removeAllColumns()` – убираем автосозданные колонки, чтобы задать свои.

`grid.addColumn(Document::getTitle).setHeader("Название")` – привязка колонки к полю сущности через method reference.

`grid.setItems(...)` – принимает коллекцию из сервиса.

Часть 6. Тестирование приложения

1. Убедитесь, что PostgreSQL запущен.
2. Запустите приложение в IntelliJ IDEA.
3. Перейдите по адресу <http://localhost:8080/documents>. Таблица будет пустой - в БД еще нет данных.
4. Откройте форму <http://localhost:8080/document-form> (из Практической работы №2) и создайте несколько документов, которые будут содержать ваши ФИО и номер группы. Они сохранятся в БД.

5. Вернитесь на страницу списка – документы должны появиться в таблице.
6. Попробуйте поиск: введите часть названия в поле – таблица должна отфильтроваться.

Предоставьте скриншоты каждого шага, а также фрагмент сохраненных данных из БД

Часть 7. Доработайте приложение

1. Добавьте сортировку колонок.
Включите сортировку для колонок «Название» и «Дата регистрации».
2. Добавьте фильтр по типу документа
Рядом с полем поиска добавьте `ComboBox<String>` с вариантами: «Все», «Приказ», «Распоряжение» и т.д.
3. Добавьте колонку «Срочный документ» в таблицу на странице `/documents`.

Содержание отчета

По итогам работы необходимо представить отчет, включающий:

1. Исходный код сущности `Document.java` с JPA-аннотациями.
2. Листинг интерфейса `DocumentRepository.java`.
3. Листинг обновленного `DocumentService.java`.
4. Фрагмент кода из `DocumentListView.java`, отвечающий за конфигурацию колонок `Grid` и логику фильтрации.
5. Скриншот окна браузера, на котором видна таблица с данными, успешно загруженными из PostgreSQL (использовать данные, созданные в прошлых практических).
6. Скриншот, демонстрирующий работу фильтра (ввод текста в поле поиска и сокращение списка в таблице).
7. Скриншоты заданий из Части 7.

Возможные проблемы и их решение

Ошибка подключения к БД. Проверьте, запущен ли PostgreSQL (docker ps), правильность URL, логина и пароля в application.properties.

Таблицы не создаются. Убедитесь, что ddl-auto=update, что сущность помечена @Entity, что Hibernate видит ее при сканировании (пакет должен быть внутри корневого пакета приложения). Если возникают другие ошибки – можно пересоздать таблицы, для этого в application.properties временно измените spring.jpa.hibernate.ddl-auto=create с update на create, перезагрузите приложение и верните обратно update.

Grid отображает пустые строки или неправильные данные. Проверьте, что updateList() вызывается и возвращает непустой список; убедитесь, что в БД есть данные.

Ошибка No property found for type. Spring Data JPA не смог распарсить имя метода. Проверьте, что имена полей в методе (findByTitleContaining...) точно совпадают с полями сущности.

Ошибка column "reg_number" does not exist. Проверьте, что имена колонок в @Column соответствуют тому, что создано в БД. При ddl-auto=update Hibernate обычно создает все сам, но если БД была создана вручную – может быть рассинхронка.

Контрольные вопросы:

1. Что такое JPA и какую роль в нем играет Hibernate?
2. Каким требованиям должен удовлетворять POJO-класс, чтобы быть JPA-сущностью?
3. Что делает аннотация @GeneratedValue(strategy = GenerationType.IDENTITY)? Какие еще стратегии генерации ID вы знаете?
4. В чем преимущество JpaRepository перед написанием JDBC-кода вручную?

5. Что такое «ленивая загрузка» (Lazy Loading) в компоненте Vaadin Grid и почему она критична для систем с миллионами документов?
6. Как реализовать сортировку по умолчанию в Grid (например, чтобы новые документы всегда были сверху)?
7. Зачем нужен слой Service, если можно вызвать Repository напрямую из UI-контроллера? Как это связано с принципами чистой архитектуры?
8. Как Spring Data JPA понимает, какой запрос выполнить для метода `findByTitleContainingIgnoreCase`?
9. В чем разница между `@Entity` и `@Table`? Когда можно не указывать `@Table`?
10. Что произойдет, если в `application.properties` указать `spring.jpa.hibernate.ddl-auto=validate`, а таблица в БД отсутствует?

Список полезных источников:

1. Официальная документация Spring, «Spring Data JPA» - <https://docs.spring.io/spring-data/jpa/reference/index.html>
2. Vaadin, «Grid» - <https://vaadin.com/docs/latest/components/grid>
3. Baeldung, «Introduction to Vaadin» - <https://www.baeldung.com/vaadin>